

IPC1 - SEGUNDO SEMESTRE 2026

Guía de Inicio: Proyecto 1

Estrategia de Arquitectura, Evaluación Inicial e Implementación Limpia del Patrón
MVC en Java Swing

 Universidad de San Carlos de Guatemala |  Facultad de Ingeniería - CYS

EVALUACIÓN PREVIA Y RESTRICCIONES

- 🚫 **Cero Colecciones Dinámicas:** Prohibido usar ArrayList, HashMap o List. Todo debe ser en arreglos estáticos y matrices.
- 📄 **Swing por Código Puro:** Prohibido el uso de diseñadores visuales (NetBeans GUI Builder, WindowBuilder, drag & drop).
- 🛡️ **Defensa con Enfoque Anti-IA:** Examen oral con modificación de código en vivo e historia de commits en GitHub.

Shelterware
ay

MOASMS)
ntrol
manage

reporting
workflow



¿POR QUÉ UN PATRÓN DE ARQUITECTURA?



Orden vs Caos

Evita el "Spaghetti Code" donde la interfaz gráfica, la validación de arreglos y la lectura de archivos están mezcladas en un solo archivo inmanejable.



Defensa y En vivo

En la defensa oral te pedirán agregar un campo o validar un estado. Si tu código es modular, sabrás exactamente en qué clase hacer el cambio sin romper la GUI.



Mantenibilidad

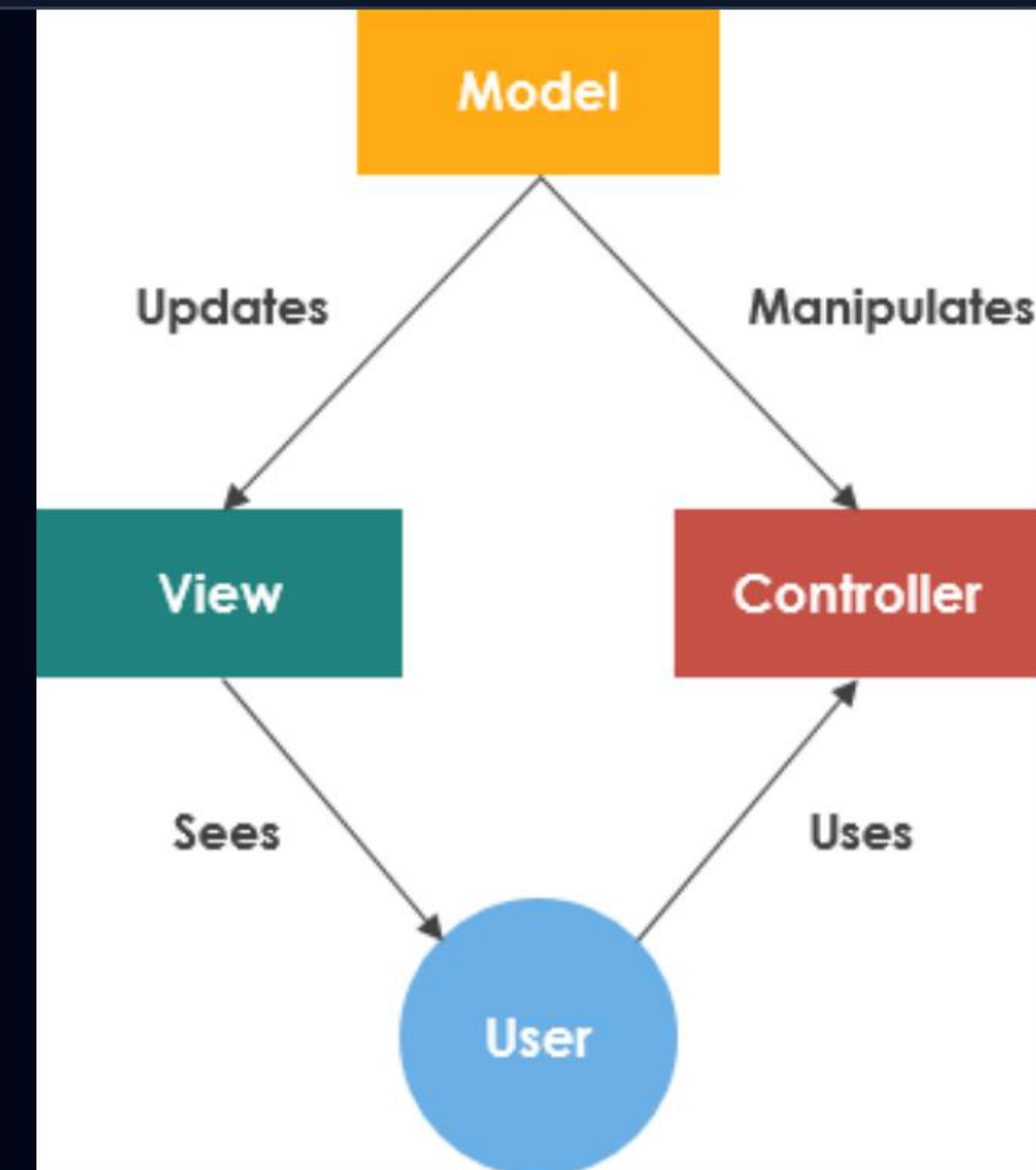
Permite modificar la forma de guardar datos o agregar reportes HTML sin necesidad de alterar los eventos de los botones de Java Swing.

RETO TÉCNICO: ARREGLOS Y MATRICES

Módulo	Estructura Permitida	Restricción Clave	Estado
Animales y Adoptantes	Arreglos estáticos de Objetos Animal[]	Tamaño fijo. Control manual mediante contador de posición e índice.	Obligatorio
Ubicaciones del Refugio	Matriz bidimensional String[filas][columnas]	Filas = Áreas del refugio, Columnas = Jaulas/Espacios físicos.	Obligatorio
Solicitudes y Rescates	Arreglos estáticos con Búsqueda Lineal	Evitar registros duplicados, validar estados y prioridades.	Obligatorio
Persistencia & Reportes	Archivos de Texto (.txt / .csv) y HTML	Generación dinámica sin librerías externas avanzadas.	Obligatorio

FUNDAMENTOS DEL PATRÓN MVC

-  **Model (Modelo):** Contiene las entidades (Animal, Adoptante) y la lógica sobre los arreglos estáticos y matrices.
-  **View (Vista):** Ventanas JFrame y paneles JPanel Swing contruidos 100% por código. Solo muestra datos y captura eventos.
-  **Controller (Controlador):** Recibe las acciones de la Vista, valida las entradas y opera sobre el Modelo.



CAPA MODELO: ESTRUCTURA DE DATOS



Entidades del Sistema

Clases POJO puras: `Animal`, `Adoptante`, `Solicitud`, `Rescates` y `Usuario`. Contienen atributos privados, constructores y métodos getters/setters sin lógica de Swing.



Gestores de Arreglos (Repor)

Clases que envuelven los arreglos estáticos: `AnimalRepository` maneja el arreglo `Animal[]`, controlando inserciones, eliminaciones lógicas, recorridos y búsquedas por ID o nombre.

CAPA VISTA: SWING POR CÓDIGO



Ventanas y Layouts

Uso explícito de BorderLayout, GridLayout y FlowLayout mediante código para construir paneles limpios e intuitivos.



Componentes Interactivos

Construcción manual de JTable, DefaultTableModel, JButton y JTextField para capturar la interacción del usuario.

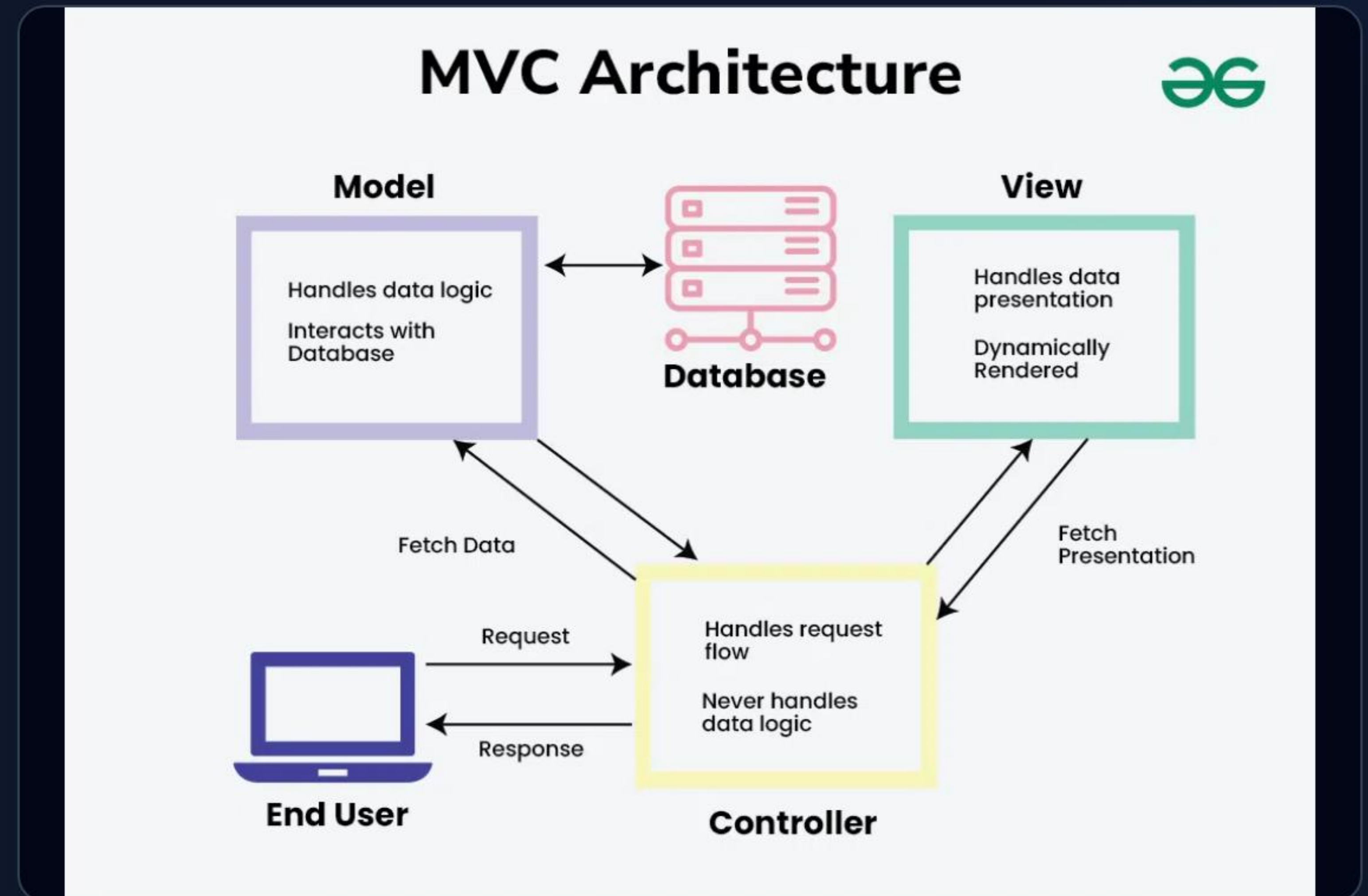


Cero Dependencia de Datos

La vista no realiza búsquedas ni valida reglas de negocio; únicamente le pasa las cadenas ingresadas al Controlador.

CAPA CONTROLADOR: LÓGICA NEGOCIO

- 🔹 **Validación de Entradas:** Verifica campos vacíos, duplicidad de IDs y formatos válidos antes de modificar el modelo.
- 📁 **Persistencia de Archivos:** Carga y guarda los datos en archivos .txt o .csv al iniciar o cerrar la app.
- 📄 **Generación de HTML:** Recorre los arreglos estáticos y genera reportes formateados en código HTML con CSS incrustado.



MVC PARA LA DEFENSA Y MODIFICACIÓN

100%

Control del Código

Facilita responder preguntas técnicas y modificar funciones vivas durante la evaluación oral.

- ✓ **Escenario Típico de Evaluación:** "Agregue un nuevo filtro por especie en la tabla de animales".
- ⚙️ **Con MVC:** Solo agregas un método de búsqueda en el Modelo, un evento en el Controlador y actualizas el `DefaultTableModel` en la Vista.
- ✗ **Sin MVC:** Tendrías que descifrar cientos de líneas mezcladas en una sola clase gigante, arriesgándote a no completar la prueba.

ESTRATEGIA DE DESARROLLO SUGERIDA

1

Paso 1: Dominio

Crear las clases de Modelo (Animal, Adoptante) y probar la lógica de arreglos estáticos en consola antes de tocar GUI.

2

Paso 2: Persistencia

Implementar lectura y escritura en archivos de texto para verificar que la información se conserve al reiniciar.

3

Paso 3: Interfaz Swing

Diseñar las ventanas Swing por código módulo por módulo, conectándolas con sus respectivos controladores.

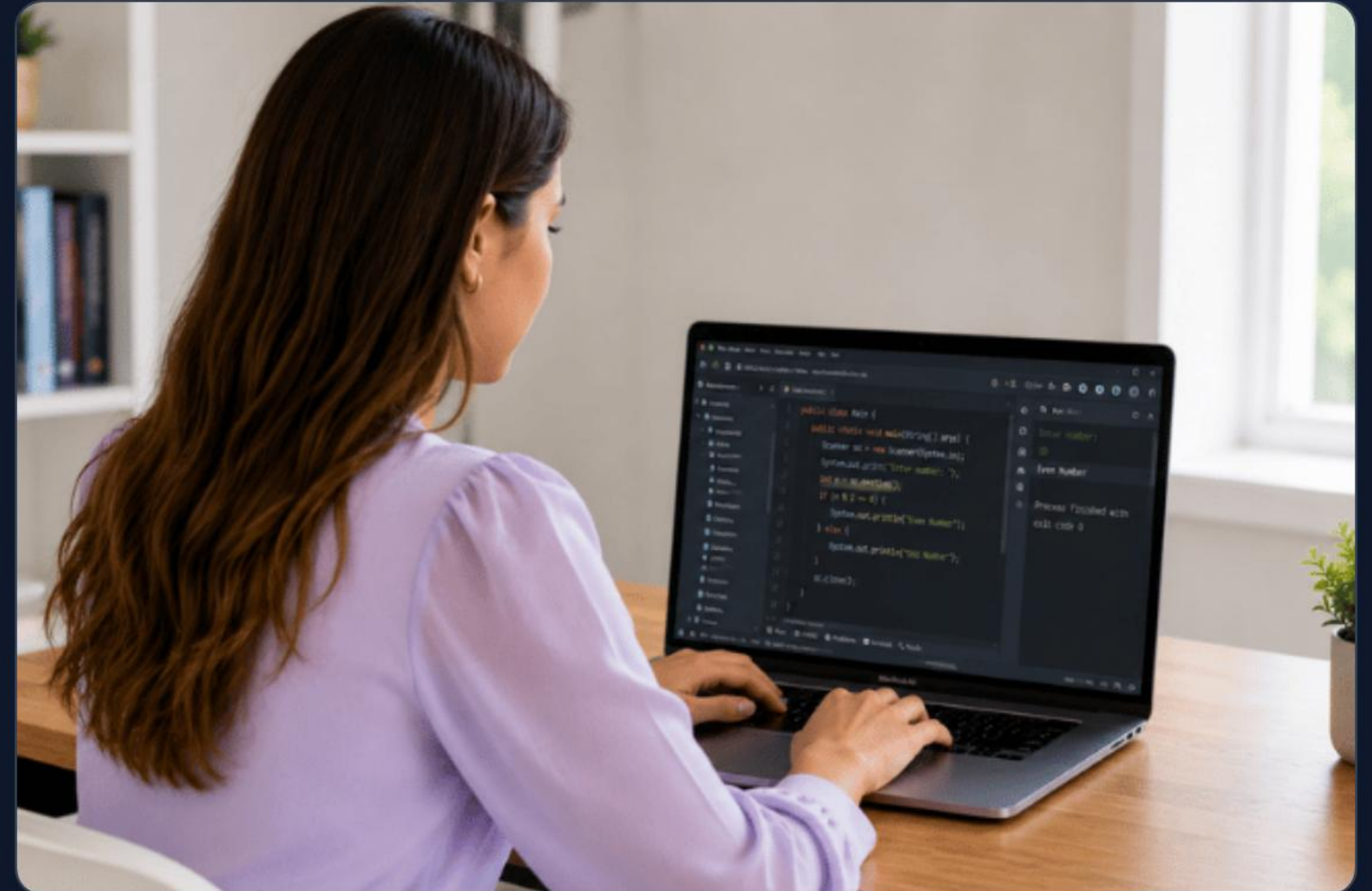
4

Paso 4: Matriz & Reportes

Construir la matriz visual del refugio y el generador de reportes HTML. Realizar pruebas integrales.

CONTROL DE VERSIONES Y BITÁCORA

-  **Commits Incrementales:** Realiza avances semanales constantes. Evita subir todo el proyecto en un único commit al final.
-  **Mensajes Semánticos:** Usa descripciones claras (ej. feat: implementa arreglo de animales, fix: corrige validacion de matriz).
-  **Bitácora Anti-IA:** Documenta decisiones clave, errores encontrados y cómo los solucionaste para acreditar tu autoría.



Claves para Alcanzar los 40 Pts

¡El secreto del éxito radica en la organización, el orden de tu arquitectura y la práctica previa!

Checklist Final para la Entrega:

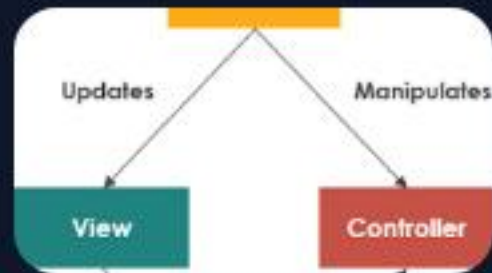
✓ Repositorio GitHub con formato correcto | ✓ Cero Collections | ✓ Swing por código | ✓ Manuales en PDF

IMAGE SOURCES



<https://multiop.com/assets/images/moasms-system-overview.jpg>

Source: multiop.com



<https://www.visual-paradigm.com/servlet/editor-content/guide/uml-unified-modeling-language/what-is-model-view-control-mvc/sites/7/2019/09/model-view-controller.png>

Source: www.visual-paradigm.com



<https://media.geeksforgeeks.org/wp-content/uploads/20240704102850/MVC-Architecture.webp>

Source: www.geeksforgeeks.org



https://tutree-main.sfo3.cdn.digitaloceanspaces.com/assets/tutor-pages/sections/1785236273383_5kuy1j6biq.png

Source: www.tutree.com